

Лекция 1. Введение в задачу языкового моделирования и история появления языковых моделей

Курс “Методы разработки программного обеспечения с использованием LLM”

Глазков Никита Олегович, AIRI | МИФИ

Сухоруков Даниил Александрович, AIRI | МИФИ

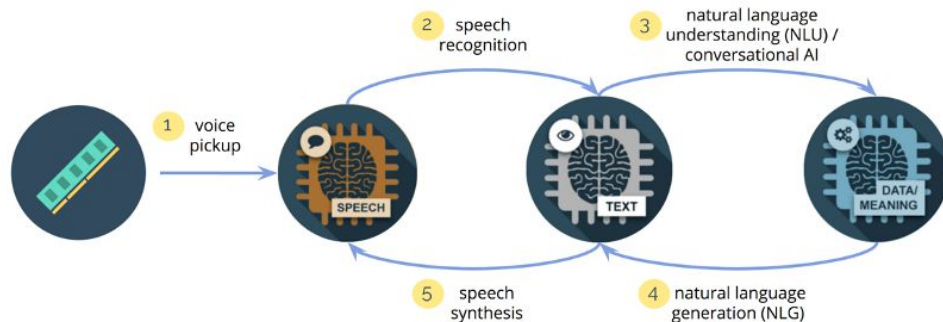
Макаров Илья Андреевич, Ph.D., AIRI | ИТМО | МИФИ

Основные цели курса

1. Научиться использовать кодогенерацию для создания проектов
2. Научиться анализировать документы с помощью краткого содержания и QA
3. Научиться разрабатывать умных ассистентов и чат-ботов
4. Выработать интуитивное понимание устройства систем, основанных на LLM
5. Научиться использовать LLM-сервисы для самостоятельного обучения

Что такое Natural Language Processing (NLP)

Обработка естественных языков - область, включающая в себя различные методы обработки и анализа текста компьютером. NLP находится на пересечении компьютерных наук, лингвистики и искусственного интеллекта.



* There may be substitutions and more complicated functionalities within NLU/conversational AI in the future.

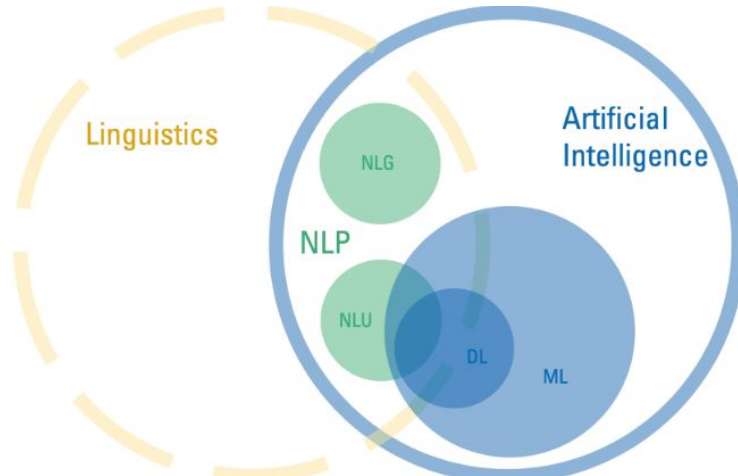
Пример работы голосового ассистента

Подразделы NLP: NLU и NLG

Natural Language Understanding (NLU) - анализ естественных языков (понимание текста)

- *Синтаксический анализ* - анализ структуры языков (например сказуемое и подлежащее)
- *Семантический анализ* - анализ значения, смысла

Natural Language Generation (NLG) - генерация естественных языков (генерация текстов)



Зачем заниматься NLP?

Цели NLP:

- Понимание структуры и значения человеческой речи
- Создание систем, способных на
 - понимание и анализ текста и речи
 - извлечение фактов и информации из текста
 - генерацию текста и речи
- Разработка алгоритмов и моделей для текста

NLP позволяет:

- Обрабатывать огромные массивы текста или речи
- Автоматизировать бизнес-процессы
- Искать информацию в массиве данных
- Решать узкие практические задачи

Основные задачи NLP (часть 1)

1. Классификация текста: Распределение текстов по категориям.

Представьте, что у вас есть гора писем от клиентов. Классификация текста поможет автоматически распределить их по темам: жалобы, вопросы о доставке, благодарности и т.д. Примеры применения: фильтрация спама, определение темы новостной статьи, автоматическая сортировка документов.

2. Анализ тональности (Sentiment Analysis): Определение эмоциональной окраски текста.

Полезно для анализа отзывов о продуктах, мониторинга социальных сетей и оценки общественного мнения. Например, можно определить, нравится ли людям новый фильм по отзывам в социальных сетях.

3. Распознавание именованных сущностей (Named Entity Recognition, NER): Выделение в тексте имён собственных, названий организаций, локаций, дат и других важных объектов.

Например, из фразы "Иван Петров работает в Сбере в Москве с 2020 года" NER извлечет сущности: "Иван Петров" (персона), "Сбер" (организация), "Москва" (локация), "2020 год" (дата). Полезно для анализа новостей, создания баз знаний и поиска информации.

4. Машинный перевод (Machine Translation): Автоматический перевод текста с одного языка на другой.

Позволяет людям общаться и получать доступ к информации на разных языках.

5. Вопрос-ответ (Question Answering): Поиск ответа на заданный вопрос в тексте или базе знаний.

Примеры: поисковые системы, чат-боты. Позволяет быстро находить нужную информацию без необходимости читать весь текст.

Основные задачи NLP (часть 2)

6. Реферирование текста (Text Summarization): Создание краткого содержания длинного текста.

Полезно для быстрого ознакомления с большим объемом информации.

7. Генерация текста (Text Generation): Автоматическое создание текстов, например, новостных статей, стихов, сценариев.

Используется в журналистике, маркетинге, развлекательной индустрии и других областях.

8. Распознавание речи (Speech Recognition) и синтез речи (Speech Synthesis): Преобразование устной речи в текст и наоборот.

Используется в голосовых помощниках, системах диктовки, озвучке текста и других приложениях.

9. Анализ зависимости (Dependency Parsing): Определение грамматических связей между словами в предложении.

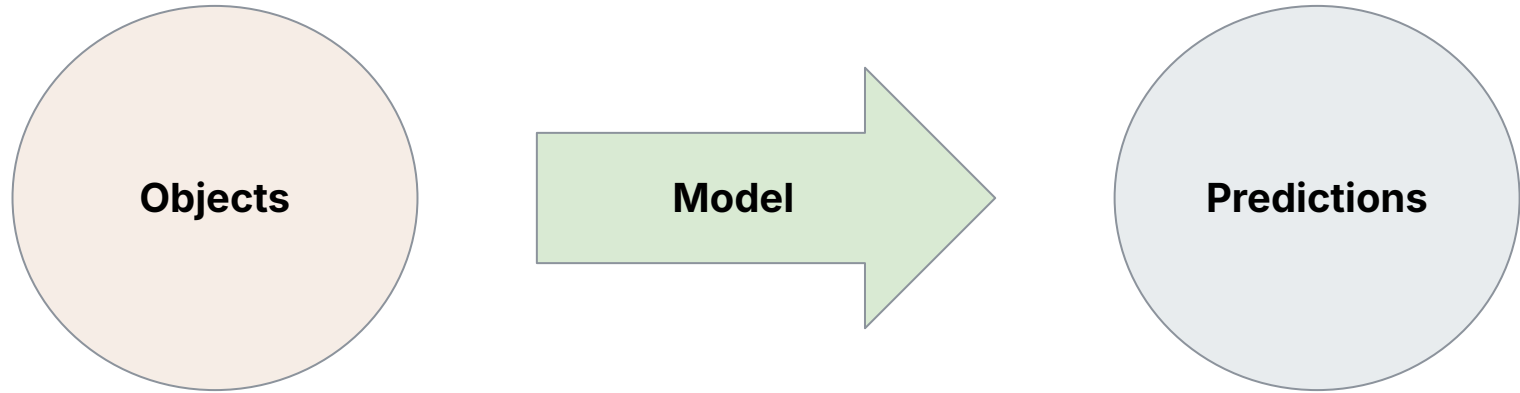
Помогает системам понять структуру предложения и отношения между словами, что важно для многих задач NLP, таких как машинный перевод и анализ тональности.

10. Извлечение отношений (Relationship Extraction): Выявление связей между сущностями в тексте.

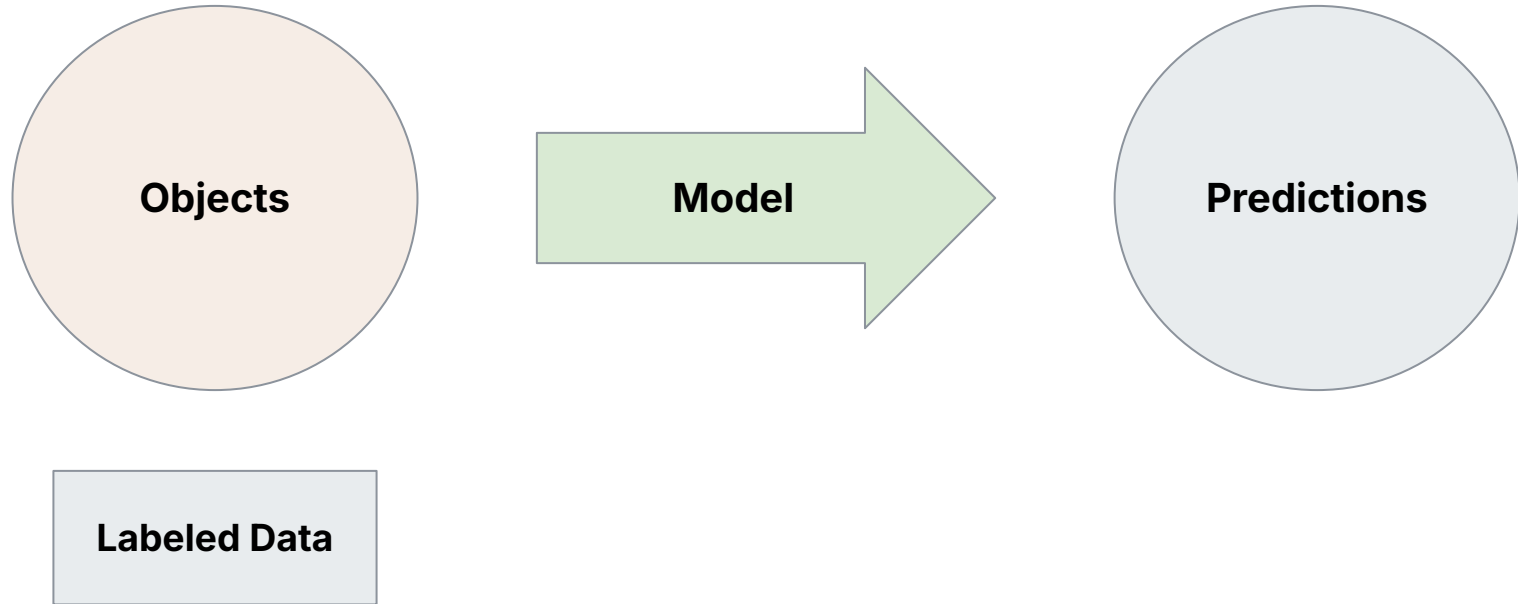
Например, из фразы "Евгений Касперский основал Лабораторию Касперского" можно извлечь отношение "основатель" между сущностями "Евгений Касперский" и "Лаборатория Касперского".

Время для вопросов

Классическая задача машинного обучения



Классическая задача машинного обучения



NLP и языковые модели

Языковые модели (LM) оценивают вероятность различных лингвистических единиц: символов, токенов, последовательностей токенов.

Web search engine / ...

I saw a cat|

I saw a cat on the chair
I saw a cat running after a dog
I saw a cat in my dream
I saw a cat book

Translation service / mail agent / ...

I saw a ca|

car ←

Keyboard / mail agent / ...

I saw a catt

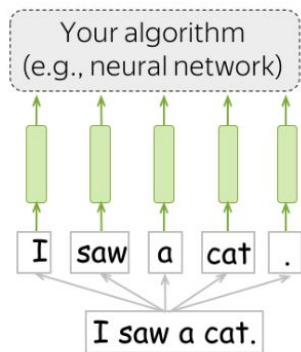
cat
car

Data

=

Text

Векторные представления слов (Word Embeddings)



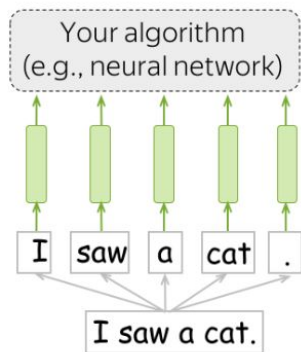
Any algorithm for solving a task

Word representation - vector
(input for your model/algorithm)

Sequence of tokens

Text (your input)

Векторные представления слов (Word Embeddings)

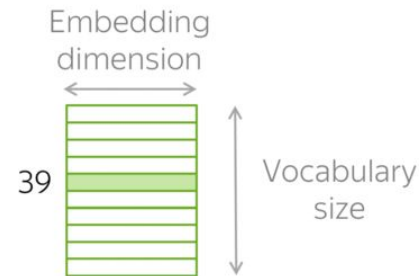


Any algorithm for solving a task

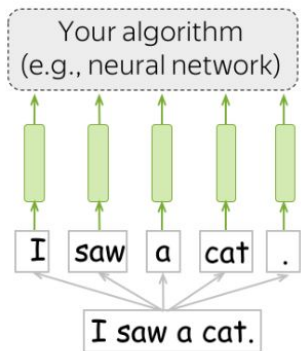
Word representation - vector
(input for your model/algorithm)

Sequence of tokens

Text (your input)



Векторные представления слов (Word Embeddings)

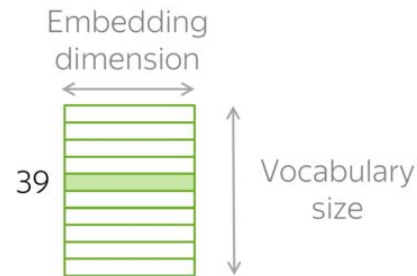


Any algorithm for solving a task

Word representation - vector
(input for your model/algorithm)

Sequence of tokens

Text (your input)

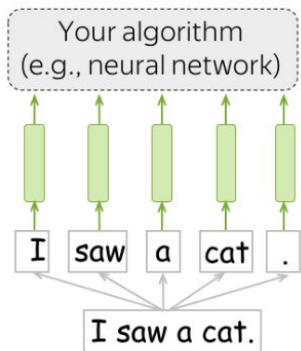


Do you know what the word **tezgüino** means ?

(We hope you do not)



Векторные представления слов (Word Embeddings)

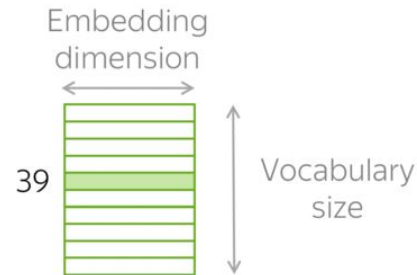


Any algorithm for solving a task

Word representation - vector
(input for your model/algorithm)

Sequence of tokens

Text (your input)



Now look how this word is used in different contexts:

A bottle of **tezgüino** is on the table.

Everyone likes **tezgüino**.

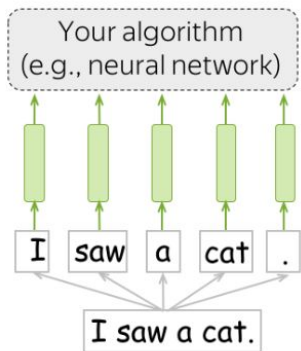
Tezgüino makes you drunk.

We make **tezgüino** out of corn.

Can you understand what **tezgüino** means ?



Векторные представления слов (Word Embeddings)

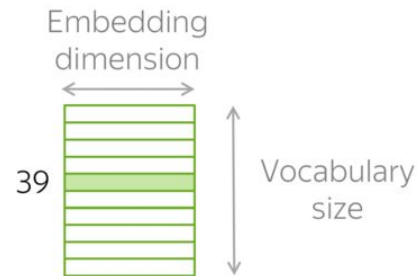


Any algorithm for solving a task

Word representation - vector
(input for your model/algorithm)

Sequence of tokens

Text (your input)



Now look how this word is used in different contexts:

A bottle of **tezgüino** is on the table.

Everyone likes **tezgüino**.

Tezgüino makes you drunk.

We make **tezgüino** out of corn.

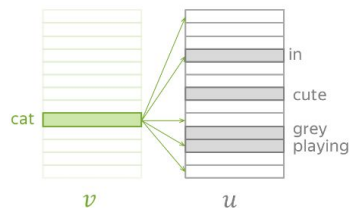
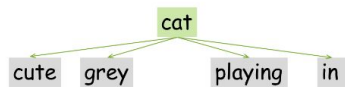
Tezgüino is a kind of alcoholic beverage made from corn.

With context, you can understand the meaning!

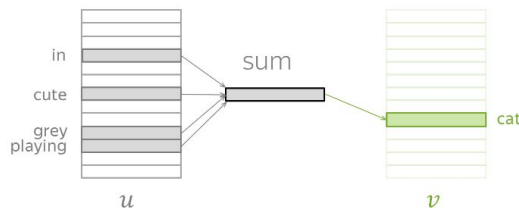
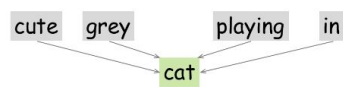


Нейронные языковые модели

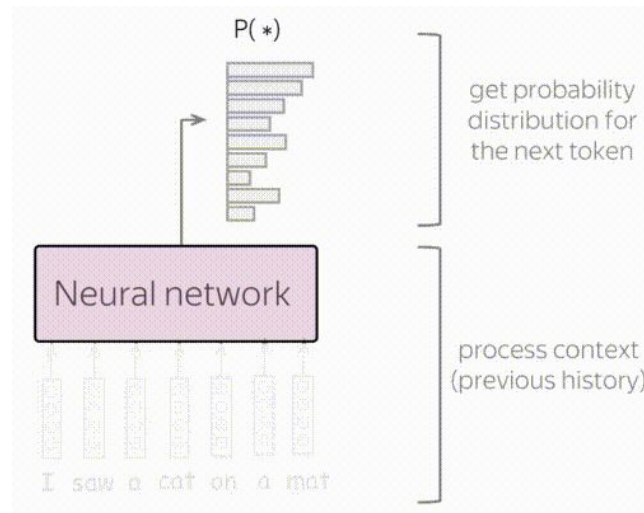
... I saw a cute grey cat playing in the garden ...



Skip-Gram: from central predict context (one at a time)



CBOW: from sum of context predict central



Нейронные языковые модели

The recipe for building large dictionaries from small ones

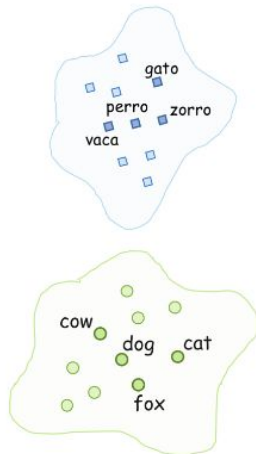
Ingredients:

- corpus in one language (e.g., **English**)
- corpus in another language (e.g., **Spanish**)
- very small dictionary

cat ↔ gato
cow ↔ vaca
dog ↔ perro
fox ↔ zorro
...

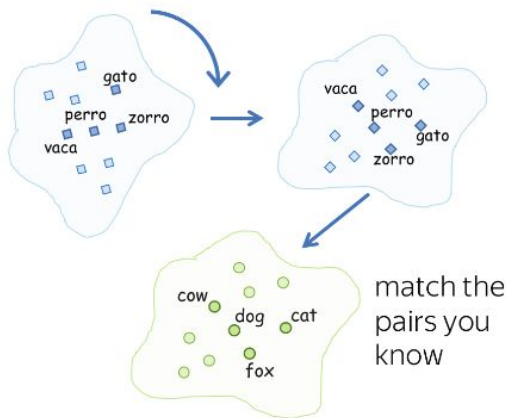
Step 1:

- train embeddings for each language



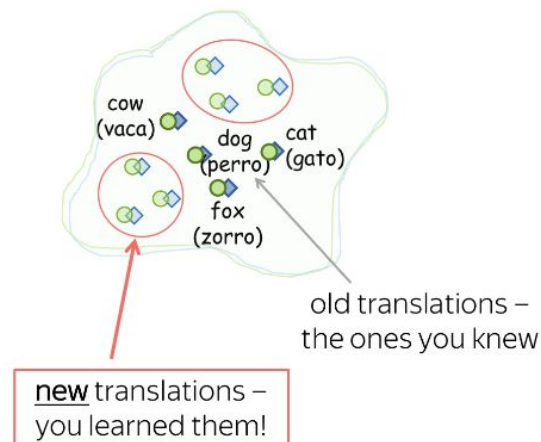
Step 2:

- linearly map one embeddings to the other to match words from the dictionary



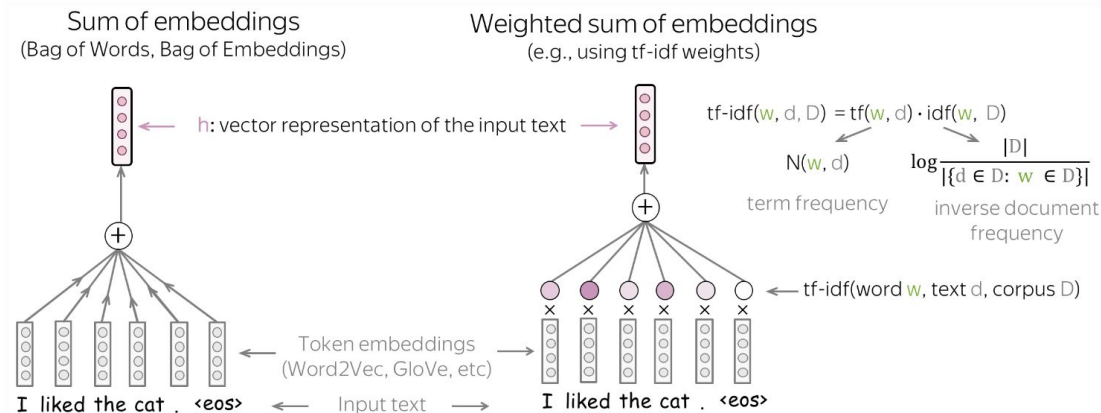
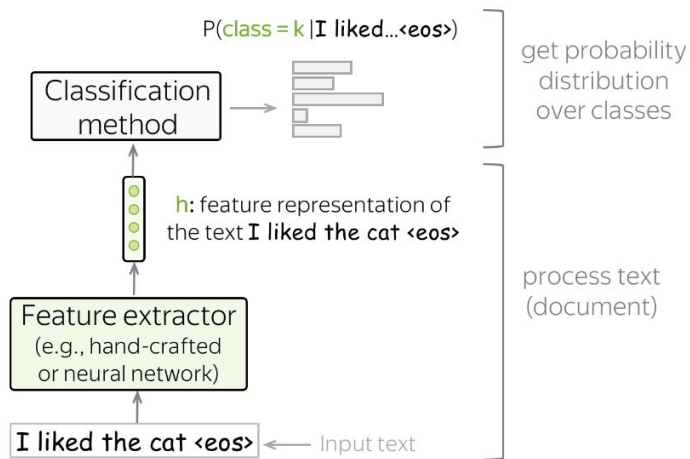
Step 3:

- after matching the two spaces, get new pairs from the new matches

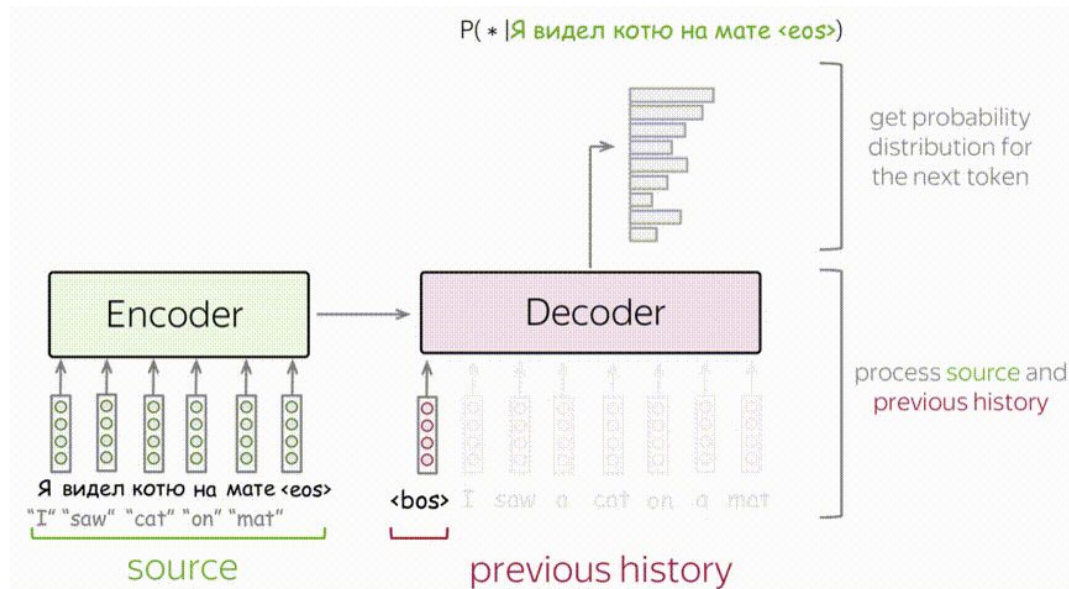
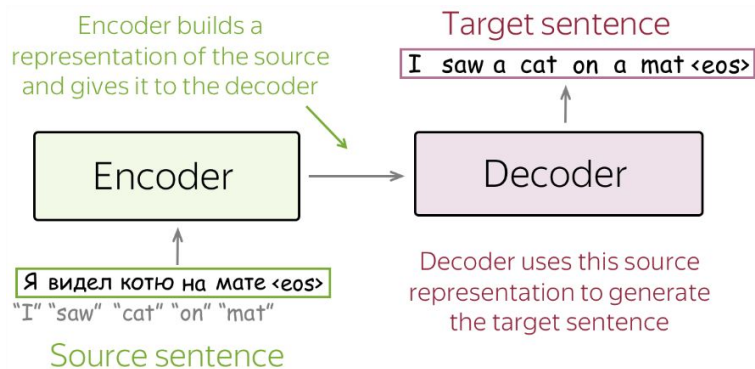


Время для вопросов

Классификация текста (Text Classification)



Encoder-Decoder



Encoder-Decoder

Encoder builds a representation of the source and gives it to the decoder

Target sentence

I saw a cat on a mat <eos>

Encoder

Decoder

Я видел котю на мате <eos>

"I" "saw" "cat" "on" "mat"

Source sentence

Decoder uses this source representation to generate the target sentence

We saw: encoder compresses the source into a single vector

Encoder

Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"

I saw a cat ...

Decoder

<bos> I saw a cat ...

Problem: this is a bottleneck!

$P(* | \text{Я видел котю на мате } \langle \text{eos} \rangle)$

Encoder

Decoder

Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"

source

<bos> I saw a cat on a mat

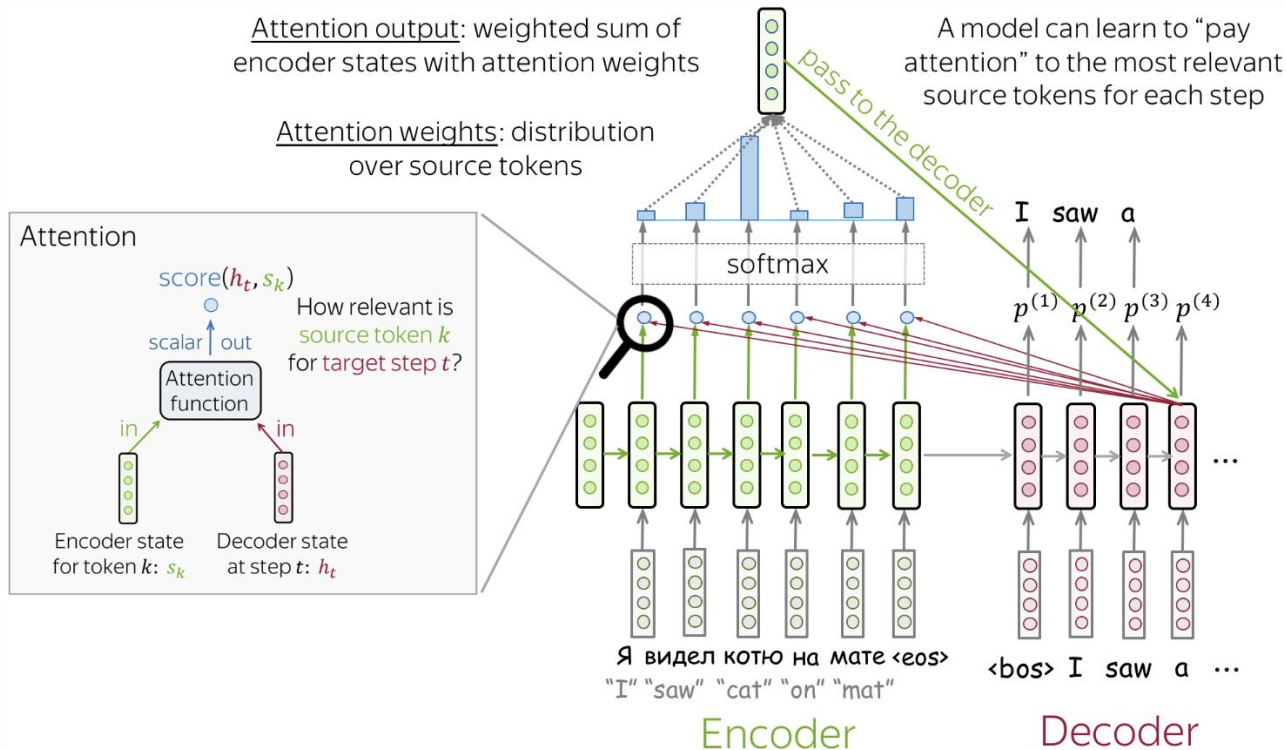
previous history

get probability distribution for the next token

process source and previous history

Время для вопросов

Attention



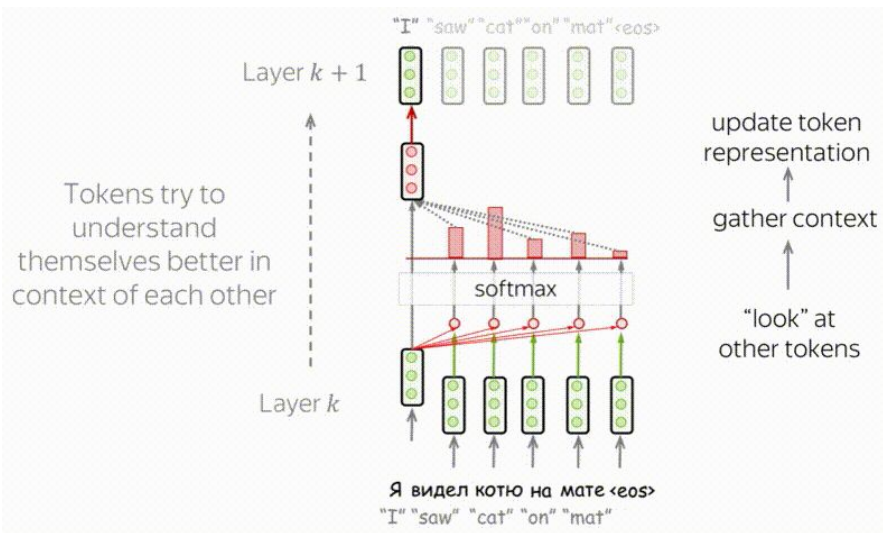
Self-Attention

Decoder-encoder attention is looking

- **from:** one current decoder state
- **at:** all encoder states

Self-attention is looking

- **from:** each state from a set of states
- **at:** all other states in the same set



I arrived at the **bank** after crossing thestreet? ...river?

What does **bank** mean in this sentence?



RNNs

$O(N)$ steps to process a sentence with length N

I've no idea: let's wait until I read the end



Transformer

Constant number of steps to process any sentence

I don't need to wait - I see all words at once!

Transformer

- **query** - asking for information
- **key** - saying that it has some information
- **value** - giving the information

Each vector receives three representations ("roles")

$$\begin{bmatrix} W_Q \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix}$$

Query: vector **from** which the attention is looking

"Hey there, do you have this information?"

$$\begin{bmatrix} W_K \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix}$$

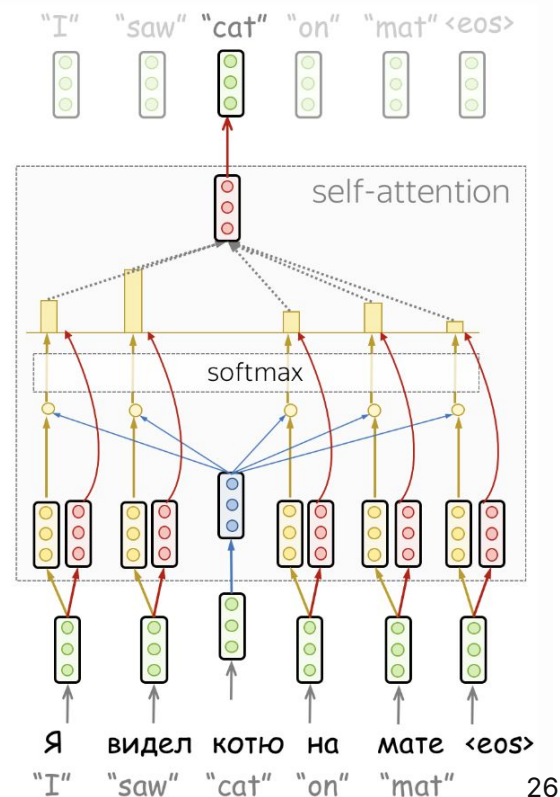
Key: vector **at** which the query looks to compute weights

"Hi, I have this information – give me a large weight!"

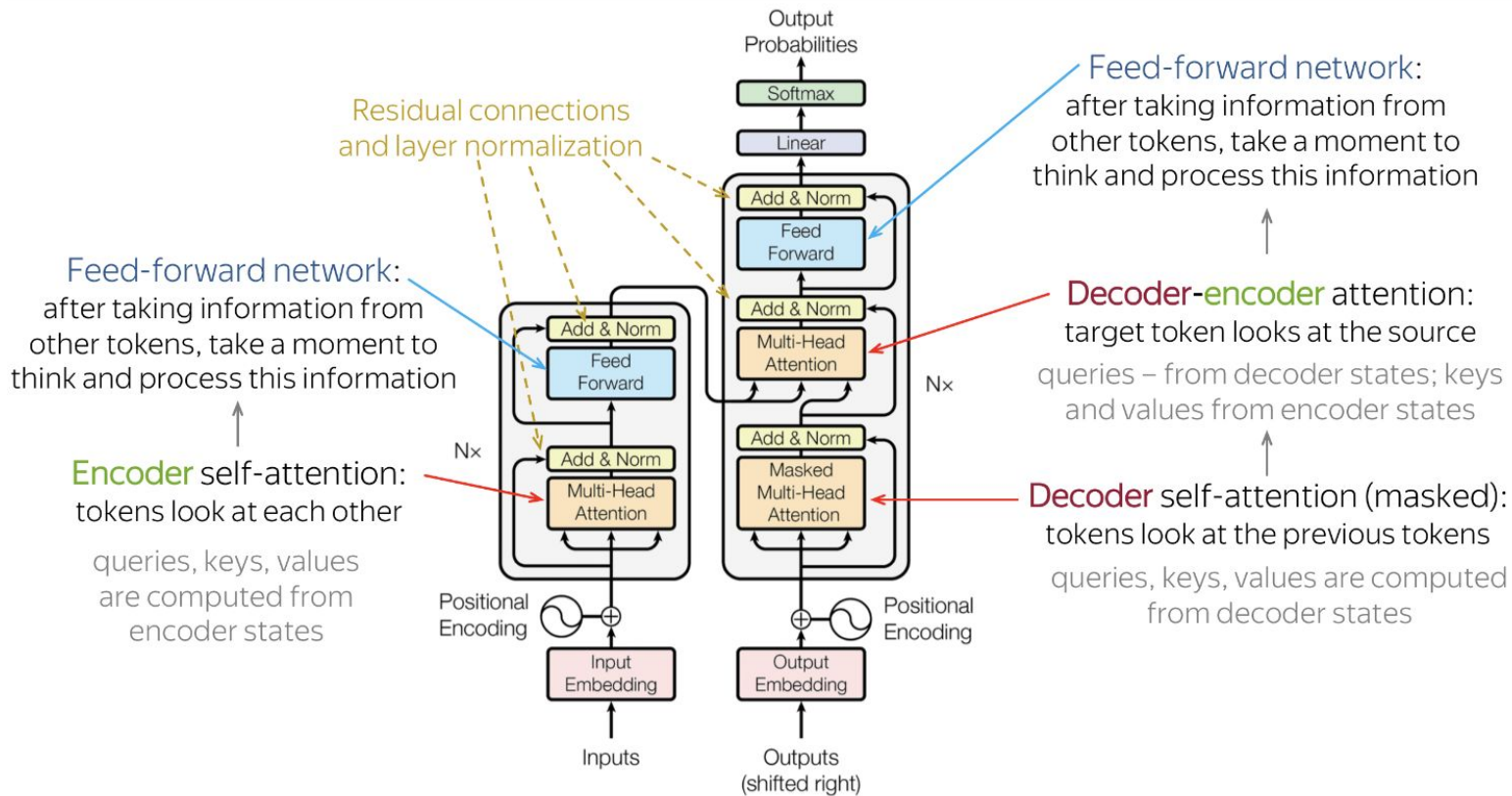
$$\begin{bmatrix} W_V \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix}$$

Value: their weighted sum is attention output

"Here's the information I have!"



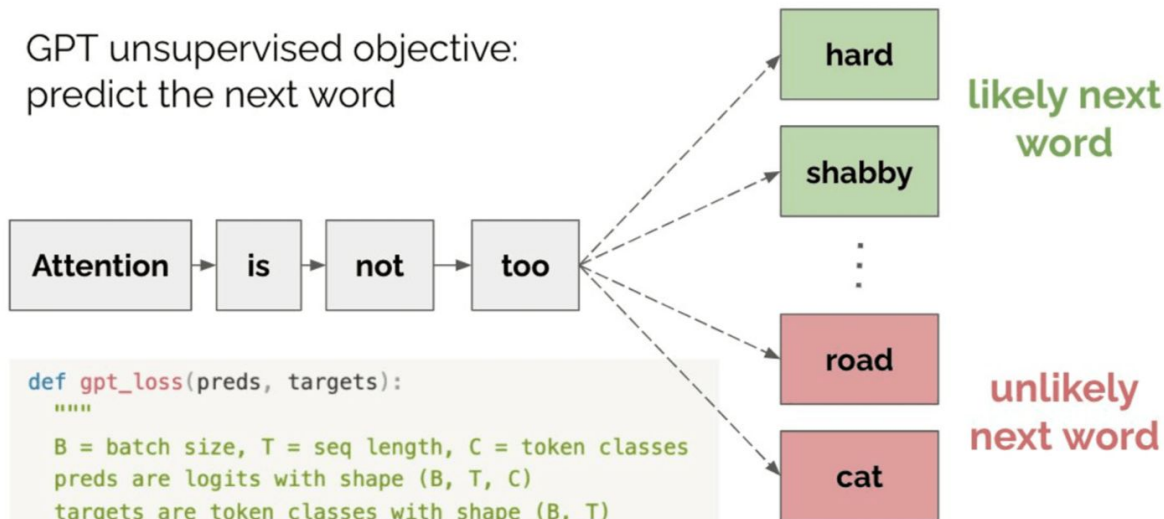
Transformer



BERT & GPT

Generative Pretrained Transformer (GPT)

GPT unsupervised objective:
predict the next word



```
def gpt_loss(preds, targets):  
    """  
    B = batch size, T = seq length, C = token classes  
    preds are logits with shape (B, T, C)  
    targets are token classes with shape (B, T)  
    """  
    preds = preds.view(B*T,C)  
    targets = targets.view(B*T)  
    return F.cross_entropy(preds, targets)
```

Время для вопросов